



POLITECNICO DI BARI

**DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELL'INFORMAZIONE
MASTER DEGREE IN AUTOMATION ENGINEERING**

Mobile robotics

Prof. Luca De Cicco

Title:

**Autonomous Navigation System for Vision-Based Human Target Following
in ROS 2 using ultralytics YOLO model**

Student:

Domenico D'Introno

ACADEMIC YEAR 2024-2025



Contents

1	Robot	3
1.1	Base Configuration	3
1.2	Enrichment with RGB-D Camera	3
2	Environment	6
2.1	Person model	6
2.2	World	8
3	YOLO Computer Vision Model	10
4	Interfaces	11
4.1	Standard Messages	11
4.2	Custom Message: CentroidCoords	11
4.3	NavigateToPose action	12
5	Custom Nodes	13
5.1	Perceptor	13
5.2	Localizer	16
5.3	Navigator	18
5.4	Initial position publisher	26
6	Launch File	29
7	Video link	31
8	Conclusion and Future Work	32

Abstract

This report details the design and implementation of a ROS 2 based system enabling a Turtle-Bot3 Waffle mobile robot to dynamically follow a human target within an environment containing obstacles. The system employs a modular architecture comprising three custom nodes. The **Perceptor** node utilizes the YOLOv8m computer vision model to detect the human target in real-time from an RGB camera feed. The **Localizer** node fuses the 2D detection coordinates with corresponding depth data from a simulated RGB-D sensor to estimate the target's 3D position relative to the robot. The **Navigator** node orchestrates the robot's movement by transforming the target's position into the global map frame, calculating appropriate goal poses, and interacting with the ROS 2 Navigation Stack via its action server. The system's effectiveness in achieving dynamic, obstacle-avoiding human following is demonstrated within the Gazebo simulation environment, utilizing custom scripts to spawn and manipulate the target model. This work showcases a practical integration of modern perception and navigation techniques for autonomous mobile robot applications like logistics, assistance, or security.

1 Robot

The mobile robot model adopted in this project is based on the TurtleBot3 Waffle model. This robot provides a compact yet versatile base equipped with differential drive, an Inertial Measurement Unit, and a 2D lidar sensor, which make it suitable for navigation and localization tasks in structured environments.

The robot model is described in the `model.sdf` sdf file, contained in the the waffle model subdirectory in the `turtlebot3_gazebo` package.

1.1 Base Configuration

The plain TurtleBot3 waffle model consists of:

- **Differential drive system**, driven by two main wheels with wheel separation of 0.287 m and radius of 0.033 m.
- **Inertial Measurement Unit (IMU)**, providing linear acceleration and angular velocity measurements with Gaussian noise injection to reproduce realistic sensor readings.
- **2D Lidar sensor**, mounted at the top of the base to provide 360-degree range scans with a maximum range of 3.5 m, used for obstacle detection and SLAM.
- **Caster wheels**, ensuring balance and stability of the platform, without adding any constraint on the robot movements.

1.2 Enrichment with RGB-D Camera

To enable 3D perception for target localization, the standard TurtleBot3 Waffle model was enhanced by incorporating a simulated RGB-D camera module, conceptually similar to devices like the Intel RealSense r200 camera module. This module, added to the robot's front structure within the simulation, provides two synchronized sensor streams:

- The **RGB camera** captures visual information at 640×480 resolution, operating at 2 Hz.
- The **Depth camera** generates pixel-aligned depth data at the same resolution and frequency. This allows to associate a 3D depth value with each pixel detected by the RGB camera.

These sensors were integrated into the simulation environment by modifying the robot's `model.sdf` file. The following snippet defines the `<link>` containing both the RGB and depth camera sensors, specifying their parameters such as update rate, resolution, field of view, output topics, and clipping distances. Notably, the `<visualize>` tag is set to `false` to reduce simulation rendering overhead, as the sensor data is primarily consumed via ROS 2 topics and not visualized in the Gazebo interface.

```
1 <link name="camera_rgb_frame">
2   <inertial>
3     <pose>0.069 -0.047 0.107 0 0 0</pose>
4     <inertia>
5       <ixx>0.001</ixx>
6       <ixy>0.000</ixy>
7       <ixz>0.000</ixz>
8       <iyy>0.001</iyy>
9       <iyz>0.000</iyz>
10      <izz>0.001</izz>
11    </inertia>
12    <mass>0.035</mass>
13  </inertial>
```

```
14
15     <pose>0.069 -0.047 0.107 0 0 0</pose>
16     <!-- Rgb camera -->
17     <sensor name="rgb_camera" type="camera">
18         <always_on>true</always_on>
19         <visualize>false</visualize>
20         <update_rate>2</update_rate>
21         <topic>camera/image_raw</topic>
22         <gz_frame_id>camera_rgb_frame</gz_frame_id>
23         <camera name="intel_realsense_r200">
24             <camera_info_topic>camera/camera_info</camera_info_topic>
25             <horizontal_fov>1.5708</horizontal_fov>
26             <image>
27                 <width>640</width>
28                 <height>480</height>
29                 <format>R8G8B8</format>
30             </image>
31             <clip>
32                 <near>0.02</near>
33                 <far>300</far>
34             </clip>
35         </camera>
36     </sensor>
37     <!-- Depth camera -->
38     <sensor name="camera_depth" type="depth">
39         <always_on>true</always_on>
40         <visualize>false</visualize>
41         <update_rate>2</update_rate>
42         <topic>camera/depth/image_raw</topic>
43         <gz_frame_id>camera_rgb_frame</gz_frame_id>
44         <camera name="intel_realsense_r200_depth">
45             <camera_info_topic>camera/depth/camera_info</camera_info_topic>
46             <horizontal_fov>1.5708</horizontal_fov>
47             <image>
48                 <width>640</width>
49                 <height>480</height>
50                 <format>R_FLOAT32</format>
51             </image>
52             <clip>
53                 <near>0.1</near>
54                 <far>10.0</far>
55             </clip>
56         </camera>
57     </sensor>
58 </link>
```

Simply adding the sensors in the SDF is insufficient; their data must be made available within the ROS 2 ecosystem. Gazebo communicates internally using Gazebo Transport, while the custom nodes operate using ROS 2 messages. The `gz_ros.bridge` allows this communication. Therefore, the bridge configuration file for the TurtleBot3 simulation, `turtlebot3_waffle_bridge.yaml` found in the `turtlebot3_gazebo` package's `param` subdirectory, required modification. Specifically, a new bridge rule was added to translate the PointCloud data generated by the depth sensor from its Gazebo Transport topic (`camera/depth/image_raw/points`) and type (`gz.msgs.PointCloudPacked`) to the corresponding ROS 2 topic (`camera/depth/points`) and standard message type (`sensor_msgs/msg/PointCloud2`). The direction: `GZ_TO_ROS` specifies that data flows from Gazebo to ROS 2.

```
1 - ros_topic_name: "camera/depth/points"  
2   gz_topic_name: "camera/depth/image_raw/points"  
3   ros_type_name: "sensor_msgs/msg/PointCloud2"  
4   gz_type_name: "gz.msgs.PointCloudPacked"  
5   direction: GZ_TO_ROS
```

In summary, the integration of the RGB-D camera upgrades the TurtleBot3 to a robot capable of perceiving the environment in 3D space, which is crucial for attaining the purpose of this project.

2 Environment

2.1 Person model

In order to test the robot capability of dynamically following a human target, an sdf model representing a person was downloaded from Gazebo fuel (<https://app.gazebosim.org/OpenRobotics/fuel/models/Standing%20person>).



Figure 1: Person model visualization in Gazebo

Then, two custom python programs have been developed to allow the model manipulation via command-line interface, enabling easy spawning and movement of the target within the simulation environment.

person_spawn.py

This script is responsible for initially placing the person model into the Gazebo simulation world at specified coordinates.

```
1 import argparse
2 import subprocess
3
4 def spawn(x, y, z):
5
6     req = (
7         f'sdf_filename: "/root/ros_workspace/person_utils/person.sdf",'
8         f'name: "my_person",'
9         f'pose: {{ position: {{ x: {x}, y: {y}, z: {z} }} }}'
10    )
11    subprocess.run([
12        "gz", "service", "-s", "/world/default/create",
13        "--reqtype", "gz.msgs.EntityFactory",
14        "--reptype", "gz.msgs.Boolean",
15        "--req", req
16    ])
17
18
19 if __name__ == "__main__":
20     parser = argparse.ArgumentParser(description="Spawn person in Gazebo")
```

```
21 parser.add_argument("--x", type=float, default=0.0, help="X position")
22 parser.add_argument("--y", type=float, default=0.0, help="Y position")
23 parser.add_argument("--z", type=float, default=0.0, help="Z position")
24
25 args = parser.parse_args()
26
27 spawn(args.x, args.y, args.z)
```

The script utilizes the `argparse` library to accept command-line arguments for the desired initial x, y, and z coordinates, with default values set to 0.0. The core functionality resides in the `spawn` function. It constructs a request string—`req`—formatted according to the `gz.msgs.EntityFactory` message type. This request specifies the path to the model's SDF file called `person.sdf`, assigns the unique name "my_person" to the spawned model within the simulation, and sets its initial pose using the provided x, y, and z coordinates.

The actual spawning is performed by invoking the Gazebo command-line tool `gz` via `subprocess.run`. Specifically, it calls the `/world/default/create` Gazebo service, providing the necessary request type `--reqtype`, reply type `--reptype`, and the constructed request message `--req`. The standard `if __name__ == "__main__":` block ensures that when the script is run, it parses the command-line arguments and passes them to the `spawn` function.

person_move.py

This script allows for the dynamic movement of the already spawned person model "my_person" to a new target location within the Gazebo world, simulating movement over a series of steps.

```
1 import argparse
2 import subprocess
3 import time
4 import re
5
6 def move(x, y, z, steps, rate):
7
8     x_start, y_start, z_start = get_person_position()
9
10    dx = (x - x_start) / steps
11    dy = (y - y_start) / steps
12    dz = (z - z_start) / steps
13
14    for i in range(steps):
15        req = f'name: "my_person", position: {{x: {x_start + dx*(i+1)}, y: {
16            ↪ y_start + dy*(i+1)}, z: {z_start + dz*(i+1)}}}'
17        subprocess.run([
18            "gz", "service", "-s", "/world/default/set_pose",
19            "--reqtype", "gz.msgs.Pose",
20            "--reptype", "gz.msgs.Boolean",
21            "--req", req
22        ])
23        time.sleep(1.0 / rate)
24
25 def get_person_position():
26
27     result = subprocess.run(
28         ["gz", "model", "-m", "my_person", "--pose"],
29         capture_output=True,
30         text=True
31     )
```



```
31 output = result.stdout
32
33 # Search for the pose using regex
34 xyz_match = re.search(r"XYZ.*:\s*\[[^\]]+\]", output)
35
36 x, y, z = [float(val) for val in xyz_match.group(1).split()]
37
38 return x, y, z
39
40
41 if __name__ == "__main__":
42     parser = argparse.ArgumentParser(description="Move person in Gazebo")
43     parser.add_argument("--x", type=float, default=0.0, help="X target")
44     parser.add_argument("--y", type=float, default=0.0, help="Y target")
45     parser.add_argument("--z", type=float, default=0.0, help="Z target")
46     parser.add_argument("--steps", type=int, default=10, help="Number of steps
47         ↪ ")
48     parser.add_argument("--rate", type=float, default=1.0, help="Hz rate")
49
50     args = parser.parse_args()
51
52     move(args.x, args.y, args.z, args.steps, args.rate)
```

Similar to the spawn script, `person_move.py` uses `argparse` to accept command-line arguments: the target `x`, `y`, and `z` coordinates, the number of discrete steps to break the movement into (`steps`, default 10), and the rate in Hz at which these steps should be executed (`rate`, default 1.0 Hz).

The `move` function orchestrates the movement. It first calls the helper function `get_person_position` to retrieve the model's current coordinates. Then, it calculates the displacement required for each step (`dx`, `dy`, `dz`) by dividing the total difference between target and start coordinates by the number of `steps`. It enters a loop that iterates `steps` times. In each iteration, it calculates the intermediate target position for that step and constructs a request string formatted according to the `gz.msgs.Pose` message type. This request specifies the model's name and the calculated intermediate position. Like the spawn script, it uses `subprocess.run` to call the Gazebo command-line tool `gz`, but this time invokes the `/world/default/set_pose` service to update the model's position. After each step, `time.sleep(1.0 / rate)` introduces a pause, controlling the overall speed of the movement.

The `get_person_position` helper function uses `subprocess.run` to execute the command `gz model -m my_person --pose`, which queries Gazebo for the current pose of the specified model. It captures the command's standard output by passing the arguments `capture_output=True`, `text=True`. The `re.search` function from the regular expression library is then used to parse this text output, searching for the line containing the XYZ coordinates and extracting the numerical values using a pattern match. These values are converted to floats and returned.

The `if __name__ == "__main__":` block handles parsing the command-line arguments and initiating the movement by calling the `move` function.

2.2 World

The simulation environment where the robot operates is defined within the `turtlebot3_gazebo` package, specifically in the `turtlebot3_world` subdirectory. The world geometry, lighting, physics, and object placements are described in the file `model.sdf`.

A key characteristic of this chosen environment is the presence of various static obstacles. These obstacles provide a challenging scenario for the robot. They necessitate that the navigation system, actively performs path planning and obstacle avoidance while dynamically pursuing the moving human target. The ability of the robot to successfully reach the target position while navigating around these obstructions is a crucial test of the overall system's effectiveness and robustness.

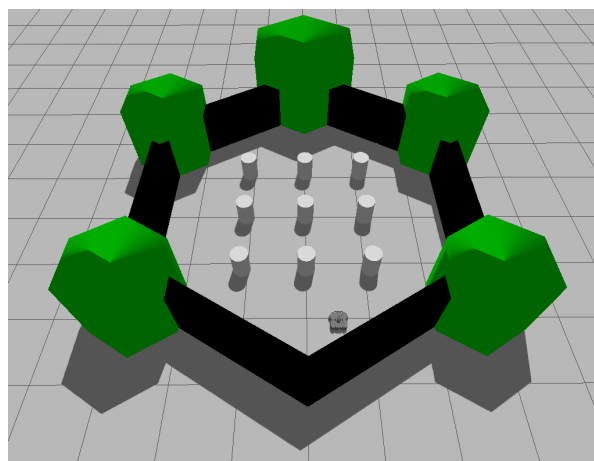


Figure 2: Environment and Turtlebot visualization in Gazebo

3 YOLO Computer Vision Model

For the task of detecting the human target within the robot's camera feed, the project utilizes the YOLO (You Only Look Once) object detection model, specifically the implementation provided by the **Ultralytics** library. YOLO is one of the most popular object detection frameworks and comprehends various models based on convolutional neural networks.

The name "You Only Look Once" refers to the fact that the algorithm requires only one forward propagation pass through the neural network to make predictions. YOLO is renowned for its efficiency and accuracy in identifying and localizing objects within images in a single pass.

The Ultralytics YOLOv8 series offers several pre-trained model variants—identified by a letter, e.g., n, s, m, l, x—which primarily differ in size, computational requirements, and detection performance. Generally, smaller models (like 'n' for nano) are faster but less accurate, while larger models (like 'x' for extra-large) achieve higher accuracy at the cost of increased inference time. These models, typically pre-trained on a well known image dataset called COCO, are capable of detecting **80 different object classes**, including 'person', which is the target for this project.

In this project, the **YOLOv8m** 'medium' variant was selected. This choice represents a deliberate compromise between inference speed and detection accuracy. While larger models might offer slightly higher precision, the 'm' variant provides a strong balance, delivering robust detection capabilities while being computationally efficient enough to run in **real-time**, especially exploiting GPU acceleration. This real-time performance is critical for the dynamic target following application, ensuring the robot receives timely updates on the target's location.

4 Interfaces

The communication among the nodes of the system relies on the ROS 2 publish/subscribe and action paradigms. Standard message types are employed for exchanging geometric and perception-related data, while a dedicated custom message has been designed to support the integration of the object detection pipeline.

4.1 Standard Messages

Several standard ROS 2 message types are used to represent positions, poses, and points in space, sometimes in a nested structure:

- **std_msgs/Header**: included in most ROS 2 messages, it is composed by two fields: a **stamp** time type field that ensures temporal synchronization of sensor data and a **frame_id** string field which provides the reference frame of the node sending data, for spatial consistency.
- **geometry_msgs/Point**: represents a point in 3D space with **float64** coordinates **x**, **y**, **z**.
- **geometry_msgs/Quaternion**: represents orientation in 3D space using four **float64** components **x**, **y**, **z**, **w**. Quaternions avoid the singularities and ambiguities of Euler angles, making them the standard choice for encoding rotations in robotics.
- **geometry_msgs/Pose**: defines a 3D pose, composed of a **position** (**geometry_msgs/Point**) and an **orientation** (**geometry_msgs/Quaternion**).
- **geometry_msgs/PointStamped**: extends the **geometry_msgs/Point** with an header for time and spatial consistency purposes. This message type is used when a specific location in space needs to be communicated, for example the 3D projection of a detected object.
- **geometry_msgs/PoseStamped**: extends the **geometry_msgs/Pose** with an header for time and spatial consistency purposes. It is used to represent navigation goals or robot localization estimates.

4.2 Custom Message: CentroidCoords

A simple custom interface has been introduced to efficiently share detections from the perception pipeline with other modules. The **CentroidCoords** message is defined as follows:

```
1 # CentroidCoords.msg
2 # Custom message interface to publish bounding box centroid pixel coordinates
3
4 std_msgs/Header header
5 int16 u
6 int16 v
```

The fields represent:

- **header**: ROS standard header containing timestamp and frame id.
- **u**: horizontal pixel coordinate of the bounding box centroid.
- **v**: vertical pixel coordinate of the bounding box centroid.

This compact representation will allow the custom nodes to communicate properly, as described later in this report.

4.3 NavigateToPose action

In addition to messages, the system makes use of **actions**, which are a special communication pattern in ROS 2 designed for tasks that may take an extended period of time to complete and require continuous feedback. An action is defined by three parts: a *goal*, a *result*, and *feedback*.

A fundamental action in the presented project is **nav2_msgs/NavigateToPose**, which allows the robot to navigate autonomously toward a specified target pose. Its interface definition is structured as follows:

- **Goal section:**
 - **geometry_msgs/PoseStamped pose**: the target pose (position and orientation in a reference frame, usually `map`) that the robot should reach.
 - **string behavior_tree**: specifies an optional behavior tree XML file that can modify the navigation strategy. Not used in this project.
- **Result section:**
 - **std_msgs/Empty result**: indicates that the action has finished. The empty message type reflects that no detailed outcome data is required beyond success or failure.
- **Feedback section:**
 - **geometry_msgs/PoseStamped current_pose**: the robot's current pose while navigating toward the goal.
 - **builtin_interfaces/Duration navigation_time**: the time already spent navigating.
 - **builtin_interfaces/Duration estimated_time_remaining**: an estimation of the remaining time before reaching the goal.
 - **int16 number_of_recoveries**: counts how many recovery behaviors (e.g., replanning, clearing costmaps) have been triggered during navigation.
 - **float32 distance_remaining**: the remaining distance to the target goal.

This structure enables the client to send navigation goals, receive continuous updates on progress, and obtain a final result once the action is completed or fails, fully exploiting the action paradigm.

5 Custom Nodes

This section describes the design and implementation of the three custom ROS 2 nodes: the **Perceptor**, the **Localizer**, and the **Navigator**. Together, they form a pipeline allowing the robot to detect a human in its camera feed, to estimate their 3D position relative to the robot, and to autonomously navigate towards that position using the Nav2 framework.

Additionally, a fourth utility node, the **Initial Position Publisher**, was developed to automate the provision of an initial pose estimate to the Nav2 localization system at startup, particularly useful for initiating the simulation reliably.

All custom ROS 2 nodes reside within the `doom_nodes` Python package. This package structure is defined by two essential files: `package.xml`, the manifest containing metadata like name, version, and crucially, build/runtime dependencies required by `colcon` to build the package and `setup.py`, the Python setup script defining installation instructions and mapping executable node names (used in launch files) to their corresponding Python main functions via `entry_points`. Together, these files enable the ROS 2 build system to correctly build, install, and launch the custom nodes.

5.1 Perceptor

The *Perceptor* node is responsible for detecting humans in the camera image stream. It leverages the YOLOv8 neural network model to perform real-time object detection.

Logic pipeline

1. **Subscribes** to the raw RGB image topic `/camera/image_raw`.
2. Applies YOLO to detect the human target.
3. Computes the centroid of the detected bounding box.
4. **Publishes** the centroid as a custom message `CentroidCoords` on `/centroid_coords`.

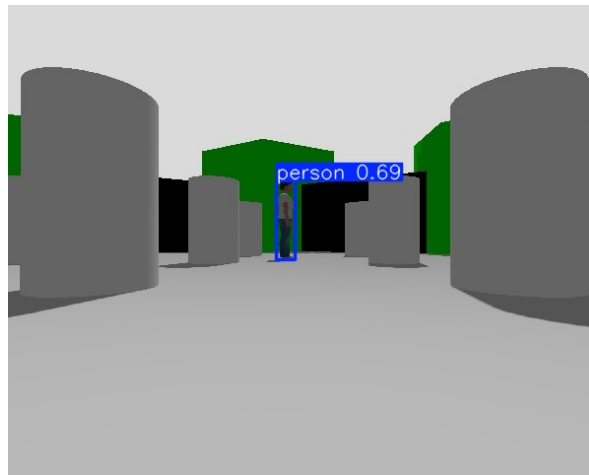


Figure 3: Turtlebot camera image with YOLO bounding box

Code analysis

```
1 import rclpy
2 from rclpy.node import Node
3
4 from sensor_msgs.msg import Image
5 from doom_interfaces.msg import CentroidCoords
6
7 from cv_bridge import CvBridge
8
9 from ultralytics import YOLO
10
11 class Detector(Node):
12
13     def __init__(self):
14         super().__init__('perceptor')
15
16         self.model = YOLO("yolov8m.pt")
17
18         self.bridge = CvBridge()
19
20         self.subscriber_ = self.create_subscription(
21             Image,
22             '/camera/image_raw',
23             self.detect,
24             10
25         )
26
27         self.publisher_ = self.create_publisher(
28             CentroidCoords,
29             '/centroid_coords',
30             10
31         )
32
33         self.get_logger().info(f"Detection node ready \n")
34
35     def detect(self, msg):
36
37         # Convert a sensor_msgs\Image message in a format that can be used by
38         # ↪ the YOLO model
39         cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='
40             ↪ passthrough')
41
42         # Detect a person (class : 0) inside the camera image, use gpu
43         results = self.model(cv_image, device="cuda", classes=[0], verbose=
44             ↪ False)
45
46         # Check that a person has actually been detected, otherwise publish
47         # ↪ the 'error centroid'
48         if len(results[0].boxes) > 0:
49             [x_min, y_min, x_max, y_max] = results[0].boxes.xyxy[0]
50
51             # Compute centroid (top 25% to avoid targeting the void space
52             # ↪ between the legs)
53             x_centroid = int((x_min + x_max) / 2)
54             y_centroid = int(y_min + (y_max-y_min) * 0.25)
55
56             centroid = CentroidCoords()
```

```
52         centroid.u = x_centroid
53         centroid.v = y_centroid
54         centroid.header = msg.header
55     else:
56         centroid = CentroidCoords()
57         centroid.u = -1
58         centroid.v = -1
59         centroid.header = msg.header
60
61     self.publisher_.publish(centroid)
62
63
64
65
66 def main(args=None):
67     rclpy.init(args=args)
68     node = Detector()
69     try:
70         rclpy.spin(node)
71     except KeyboardInterrupt:
72         pass
73     finally:
74         node.destroy_node()
75         rclpy.shutdown()
76
77
78 if __name__ == '__main__':
79     main()
```

The class **constructor** initializes the **perceptor** node: it loads the YOLOv8 model, initializes the CV bridge, which converts between ROS2 Image messages and OpenCV images, subscribes to camera topic, and sets up a publisher for the detected centroid coordinates.

The **detect** callback function runs person detection on the received camera image using YOLO, extracts the bounding box centroid, and publishes it as **CentroidCoords**. Note that the **model** function takes as arguments **device="cuda"** to specify to use the GPU when running the model, plus, the **classes=[0]** argument narrows the research to just persons.

An important design choice is to compute the person pixel centroid as the **top 25% of its silhouette**, fixing a bug where inconsistent depth measurements and mislocalization occurred when the centroid fell between the legs.

Also, if no person is detected, the node publishes an **'error centroid'**, meaning a centroid with both pixels coordinates set to -1. This strategy allows the downstream nodes to identify target loss. Finally, the script includes the standard Python entry point structure to make it executable as a ROS 2 node.

The **main** function handles the node's lifecycle. It begins by initializing the ROS 2 client library using **rclpy.init()**. Then, it instantiates the **Detector** node object. The core of the function is the **try/except/finally** block: it enters the ROS 2 event loop, keeping the node alive to receive messages and execute callbacks until the node is shut down by the user, then it ensures graceful shutdown: upon interruption, the **node.destroy_node()** method is called to release ROS 2 resources, followed by **rclpy.shutdown()** to close the client library communication.

The **if __name__ == '__main__':** check is a standard Python idiom. It ensures that the **main()** function is called only when the script is executed directly.

5.2 Localizer

The *Localizer* node acts as the bridge between 2D image perception and the 3D world. Its primary role is to estimate the 3D position of the detected human target in the camera's coordinate frame by combining the pixel coordinates from the *Perceptor* with depth information from the robot's depth camera.

Logic pipeline

1. **Subscribes** to the centroid coordinates topic `/centroid.coords` published by *Perceptor*.
2. **Subscribes** to the depth point cloud topic `/camera/depth/points`.
3. Uses an **Approximate Time Synchronizer** to receive pairs of centroid and point cloud messages that are close in time.
4. In the callback, checks if the received centroid is valid or an 'error centroid'.
5. If the centroid is valid, it extracts the corresponding 3D point (x, y, z) from the point cloud using the pixel coordinates (u, v) to compute the corresponding index.
6. If the centroid is an 'error centroid', it generates a predefined 'error point'.
7. **Publishes** the resulting 3D point (either valid or error) as a `geometry_msgs/PointStamped` message on the `/target.position.camera` topic.

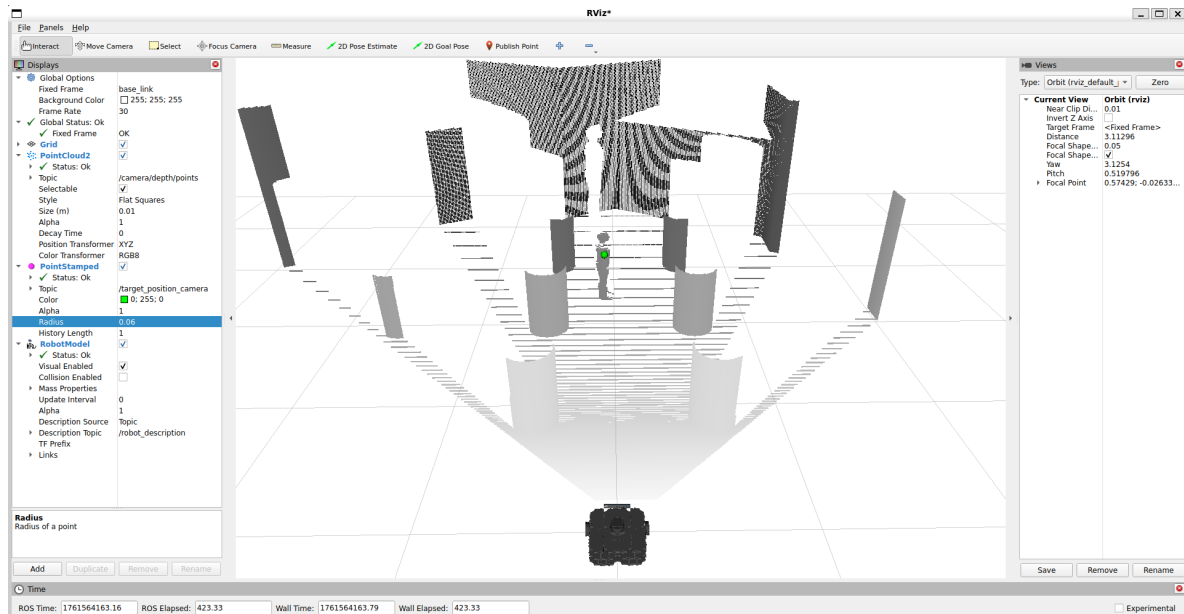


Figure 4: Rviz2 interface showing the turtlebot model, the depth pointcloud and the pointstamped published by the Localizer node

Code analysis

```
1 import rclpy
2 from rclpy.node import Node
3
4 from sensor_msgs.msg import PointCloud2
5 from geometry_msgs.msg import PointStamped
6 from doom_interfaces.msg import CentroidCoords
7
8 from message_filters import Subscriber, ApproximateTimeSynchronizer
9 import sensor_msgs_py.point_cloud2 as pc2
10
11 import numpy as np
12
13
14 class Localizer(Node):
15
16     def __init__(self):
17         super().__init__('localizer')
18
19         self.sub_centroid = Subscriber(self, CentroidCoords, '/centroid_coords
20         ↪ ')
21         self.sub_depth = Subscriber(self, PointCloud2, '/camera/depth/points')
22
23         # Initialize approximate time synchronizer to synchronize the two
24         ↪ different messages
25         ats = ApproximateTimeSynchronizer(
26             [self.sub_centroid, self.sub_depth],
27             queue_size=10,
28             slop=0.1
29         )
30         ats.registerCallback(self.localize)
31
32         self.publisher_ = self.create_publisher(
33             PointStamped,
34             '/target_position_camera',
35             10
36         )
37
38         self.get_logger().info(f"Localization node ready \n")
39
40     def localize(self, msg_centroid, msg_depth):
41
42         u = msg_centroid.u
43         v = msg_centroid.v
44
45         # If the 'error centroid' has been received, forward the 'error point
46         ↪ ', otherwise
47         # normally extract the depth point from the pointcloud
48         if u < 0 or v < 0:
49             x = 0.0
50             y = 0.0
51             z = -1.0
52         else:
53             height = msg_depth.height
54             width = msg_depth.width
55             # The pointcloud has linear indexing
56             index = v * width + u
```

```
54         points_array = pc2.read_points_numpy(msg_depth, field_names=("x",
55             ↪ "y", "z"))
56         x, y, z = points_array[index]
57
58         msg = PointStamped()
59         msg.point.x = float(x)
60         msg.point.y = float(y)
61         msg.point.z = float(z)
62         msg.header = msg_depth.header
63
64         self.publisher_.publish(msg)
65
66
67
68 def main(args=None):
69     rclpy.init(args=args)
70     node = Localizer()
71     try:
72         rclpy.spin(node)
73     except KeyboardInterrupt:
74         pass
75     finally:
76         node.destroy_node()
77         rclpy.shutdown()
78
79
80 if __name__ == '__main__':
81     main()
```

The class **constructor** initializes the **localizer** node. It sets up two **message_filters.Subscriber** instances, one for the **CentroidCoords** messages from the *Perceptor* and another for the **PointCloud2** messages from the depth sensor. Crucially, it initializes an **ApproximateTimeSynchronizer**. This synchronizer ensures that the **localize** callback function is only called when a pair of centroid and point cloud messages with closely matching timestamps—within the **slop** parameter value, here 0.1 seconds—are received. This synchronization is vital for accurately associating a 2D detection with its corresponding 3D depth data. Finally, the constructor creates a publisher for the resulting **PointStamped** message.

The **localize** callback is the core logic. It receives the synchronized centroid and point cloud messages. It first checks if the centroid coordinates **u** and **v** are negative, which signifies the 'error centroid' sent by the *Perceptor* when no person is detected: in this case, it constructs and publishes a **PointStamped** message with **z = -1.0** to propagate this error signal downstream. If the centroid is valid, it calculates the linear **index** corresponding to the pixel coordinates within the point cloud structure (using **v * width + u**). It then uses the **pc2.read_points_numpy** function to read the point cloud into a NumPy array and extracts the single 3D point coordinates at the calculated **index**. Finally, it constructs a **PointStamped** message using the determined coordinates—either valid 3D point or error point—sets its header using the timestamp from the input **msg_depth** for consistency, and publishes it.

Finally, the script includes the usual Python entry point structure.

5.3 Navigator

The *Navigator* node is the central controller responsible for the robot's autonomous movement. It orchestrates the dynamic target following behavior by processing the 3D target position provided by

the *Localizer*, determining the appropriate goal pose, and interacting with the ROS 2 Navigation Stack Nav2 via its Action interface to command the robot.

Logic pipeline

1. **Subscribes** to the target position topic `/target_position_camera` published by *Localizer*.
2. Uses a **Timer** to periodically trigger the main logic.
3. The timer callback `navigate`:
 - Handles the 'error point', which indicates **target loss**, by canceling any active Nav2 goal and resetting the internal state.
 - Combines the target position and the robot current position to **compute the next potential goal pose** and the relative straight-line distance.
 - If the **next goal is too close**, then the robot will only rotate towards the goal, aiming at the target, but avoiding getting any closer, in order to keep a safety distance with the person.
 - If a **goal is already in progress**:
 - If the computed next goal is significantly different from the current one, then cancels the current goal and sets the new one.
 - Otherwise, if the goals are too similar, does nothing, maintaining the current goal that Nav2 was already following.
 - If no goal is in progress, simply sends the computed goal to Nav2.

Code analysis

```
1 import rclpy
2 from rclpy.node import Node
3 from rclpy.time import Time
4 from rclpy.duration import Duration
5 from rclpy.action import ActionClient
6
7 import math
8
9 from geometry_msgs.msg import PointStamped, PoseStamped, Quaternion
10 from tf2_geometry_msgs.tf2_geometry_msgs import do_transform_point
11
12 from tf2_ros.buffer import Buffer
13 from tf2_ros.transform_listener import TransformListener
14 from tf2_ros import TransformException
15
16 from nav2_msgs.action import NavigateToPose
17 from action_msgs.msg import GoalStatus
18
19
20 class Navigator(Node):
21
22     def __init__(self):
23         super().__init__('navigator')
24
25         self.tf_buffer = Buffer()
26         self.tf_listener = TransformListener(self.tf_buffer, self)
```

```
27
28     # Transform-related variables
29     self.camera_frame = 'camera_rgb_frame'
30     self.robot_frame = 'base_link'
31     self.global_frame = 'map'
32     self.transform_timeout = rclpy.duration.Duration(seconds=0.5)
33
34     # Null point definition, utils
35     self.null_point = PointStamped()
36     self.null_point.point.x = 0.0
37     self.null_point.point.y = 0.0
38     self.null_point.point.z = 0.0
39
40     # Node 'internal state'
41     self.person_position_camera = None
42     self.goal_in_progress = False
43     self.current_goal_handle = None
44     self.current_goal = None
45
46     # Safety distance from the target
47     self.stopping_distance = 0.75
48
49     # Update tolerances
50     self.distance_update_threshold = 0.05
51     self.yaw_update_threshold = 0.05
52
53     self.subscriber_ = self.create_subscription(
54         PointStamped,
55         '/target_position_camera',
56         self.save_point,
57         10
58     )
59
60     self._action_client = ActionClient(self, NavigateToPose, '
    ↪ navigate_to_pose')
61
62     self.timer = self.create_timer(1.0, self.navigate)
63
64     self.get_logger().info(f"Navigator node ready \n")
65
66     def save_point(self, msg):
67
68         self.person_position_camera = msg
69
70     def navigate(self):
71
72         # Check if the target point has already been published
73         if self.person_position_camera is None:
74             return
75
76         # Check if the 'error point' has been received, in this case stop the
77         ↪ robot
78         if self.person_position_camera.point.z == -1.0:
79             self.get_logger().warn("Target lost. Stopping robot.")
80             if self.goal_in_progress and self.current_goal_handle:
81                 self.current_goal_handle.cancel_goal_async()
```

```
81         self.goal_in_progress = False
82         self.current_goal_handle = None
83         self.current_goal = None
84         return
85
86         # Transform the target point from the robot camera frame to the global
87         ↪ map frame
88     try:
89         t = self.tf_buffer.lookup_transform(
90             self.global_frame,
91             self.camera_frame,
92             self.person_position_camera.header.stamp,
93             timeout=self.transform_timeout
94         )
95         person_position = do_transform_point(self.person_position_camera,
96             ↪ t)
97     except TransformException as ex:
98         return
99
100     # Retrieve robot current position wrt the global map frame
101     robot_position = self.get_robot_position(self.global_frame)
102     if robot_position is None:
103         self.get_logger().warn("Could not get robot position, skipping
104             ↪ navigation.")
105     return
106
107     # Compute the new goal, goal SLD distance
108     self.next_goal, goal_distance = self.compute_goal(robot_position,
109         ↪ person_position)
110
111     # If the SLD to the goal is less than a threshold, just rotate towards
112     ↪ the goal
113     if goal_distance < self.stopping_distance:
114         self.next_goal.pose.position = robot_position.point
115
116     # Reset the goal if one goal was already up, otherwise simply set the
117     ↪ goal
118     if self.goal_in_progress:
119         # Skip the goal reset if the new goal is similar to the old one
120         if not self.are_goals_different(self.current_goal, self.next_goal)
121             ↪ :
122             return
123         # Cancel the previous goal
124         cancel_future = self.current_goal_handle.cancel_goal_async()
125         # Reset the goal as soon as it's cancelled using the callback
126         cancel_future.add_done_callback(self.goal_canceled_callback)
127     else:
128         self.send_goal(self.next_goal)
129
130     def are_goals_different(self, old_goal, new_goal):
131
132         # If one of the two goals does not exist, then the new goal of course
133         ↪ is different
134         if old_goal is None or new_goal is None:
135             return True
136
137         # If both goals exist, check if they are different
138         if old_goal.pose.position != new_goal.pose.position:
139             return True
140         if old_goal.pose.orientation != new_goal.pose.orientation:
141             return True
142         return False
```

```
129     # Check linear distance between goals
130     dist = math.hypot(
131         new_goal.pose.position.x - old_goal.pose.position.x,
132         new_goal.pose.position.y - old_goal.pose.position.y
133     )
134     if dist > self.distance_update_threshold:
135         return True
136
137     # Check angular distance between goals
138     old_yaw = self.quaternion_to_yaw(old_goal.pose.orientation)
139     new_yaw = self.quaternion_to_yaw(new_goal.pose.orientation)
140     yaw_diff = abs(self.normalize_angle(old_yaw - new_yaw))
141     if yaw_diff > self.yaw_update_threshold:
142         return True
143
144     # If none of the previous conditions is satisfied, then the new goal
145     # ↪ is very close to the old one
146     return False
147
148 def get_robot_position(self, source_frame):
149     try:
150         t = self.tf_buffer.lookup_transform(
151             source_frame,
152             self.robot_frame,
153             Time(),
154             self.transform_timeout
155         )
156         # The transform uses the null point since the robot in its own
157         # ↪ frame is located of course in
158         # the null coordinates
159         self.null_point.header.frame_id = self.robot_frame
160         self.null_point.header.stamp = Time().to_msg()
161         robot_position = do_transform_point(self.null_point, t)
162         return robot_position
163     except TransformException as ex:
164         return
165
166 def compute_goal(self, robot_position, target_position):
167
168     robot_x = robot_position.point.x
169     robot_y = robot_position.point.y
170     target_x = target_position.point.x
171     target_y = target_position.point.y
172     dx = target_x - robot_x
173     dy = target_y - robot_y
174
175     goal_distance = math.hypot(dx, dy)
176
177     yaw = math.atan2(dy, dx)
178     q = self.yaw_to_quaternion(yaw)
179
180     goal_pose = PoseStamped()
181     goal_pose.pose.position.x = target_x
182     goal_pose.pose.position.y = target_y
183     goal_pose.pose.position.z = 0.0
```

```
183     goal_pose.pose.orientation = q
184     goal_pose.header.frame_id = self.global_frame
185     goal_pose.header.stamp = self.get_clock().now().to_msg()
186
187     return goal_pose, goal_distance
188
189     def goal_canceled_callback(self, future):
190
191         # If the goal has been successfully canceled, send the new_goal and
192         # ↪ reset the state,
193         # otherwise simply reset the state
194         try:
195
196             cancel_response = future.result()
197             self.goal_in_progress = False
198             self.current_goal_handle = None
199             self.current_goal = None
200             self.send_goal(self.next_goal)
201         except:
202             self.goal_in_progress = False
203             self.current_goal_handle = None
204             self.current_goal = None
205
206     def send_goal(self, goal_pose):
207
208         goal_msg = NavigateToPose.Goal()
209         goal_msg.pose = goal_pose
210
211         self.get_logger().info(f'Sending goal: {goal_pose.pose.position.x}, {
212         ↪ goal_pose.pose.position.y}')
213
214         self._action_client.wait_for_server()
215
216         # Asynchronously send goal
217         self._send_goal_future = self._action_client.send_goal_async(goal_msg)
218
219         # Add callback to be executed when the goal request has been served
220         self._send_goal_future.add_done_callback(self.goal_response_callback)
221
222     def goal_response_callback(self, future):
223
224         goal_handle = future.result()
225
226         # If the goal has not been accepted, log and reset the internal state
227         if not goal_handle.accepted:
228             self.get_logger().warn('Goal rejected')
229             self.goal_in_progress = False
230             self.current_goal_handle = None
231             self.current_goal = None
232             return
233
234         # If the goal has not been accepted, log and set the internal state
235         self.get_logger().info('Goal accepted')
236         self.goal_in_progress = True
237         self.current_goal_handle = goal_handle
238         self.current_goal = self.next_goal
```



```
237
238     # Asynchronously ask for navigation results
239     self._get_result_future = goal_handle.get_result_async()
240
241     # Add callback to be executed when the results request has been served
242     self._get_result_future.add_done_callback(self.get_result_callback)
243
244     def get_result_callback(self, future):
245
246         self.goal_in_progress = False
247         self.current_goal_handle = None
248         self.current_goal = None
249
250     @staticmethod
251     def yaw_to_quaternion(yaw):
252
253         q = Quaternion()
254         q.w = math.cos(yaw / 2.0)
255         q.z = math.sin(yaw / 2.0)
256         q.x = 0.0
257         q.y = 0.0
258         return q
259
260     @staticmethod
261     def quaternion_to_yaw(q):
262
263         yaw = 2.0 * math.atan2(q.z, q.w)
264         return yaw
265
266     @staticmethod
267     def normalize_angle(angle):
268
269         while angle > math.pi:
270             angle -= 2.0 * math.pi
271         while angle < -math.pi:
272             angle += 2.0 * math.pi
273         return angle
274
275
276
277
278     def main(args=None):
279         rclpy.init(args=args)
280         node = Navigator()
281         try:
282             rclpy.spin(node)
283         except KeyboardInterrupt:
284             pass
285         finally:
286             node.destroy_node()
287             rclpy.shutdown()
288
289
290     if __name__ == '__main__':
291         main()
```

The class **constructor** initializes the navigator node. It sets up the necessary tf2 buffer and

listener for coordinate transformations, defining key frame names —`camera_frame`, `robot_frame` and `global_frame`— and a transformation timeout. It then initializes the node state variables: `person_position_camera` to store the latest target position from the subscriber, `goal_in_progress`, a boolean flag to track if a goal is currently being executed, `current_goal_handle` to store the action server goal handle and `current_goal` to store the actual `PoseStamped` message corresponding to the goal Nav2 is currently pursuing. Thresholds are defined: `stopping_distance` determines the safety distance, while `distance_update_threshold` and `yaw_update_threshold` control the sensitivity for goal updates. A subscriber is created for the `/target_position_camera` topic, using the simple `save_point` callback. The Action Client `_action_client` is initialized to communicate with the Nav2 `navigate_to_pose` action server. Finally, a **Timer** is created to periodically execute the main `navigate` method every 1.0 second.

The `save_point` callback simply saves the target position in the camera frame, received from the localizer node, in the node state.

The primary logic resides in the `navigate` method. Triggered by the timer, it first checks if the target point `person_position_camera` has been received. It then checks if the received point is an 'error point' by inspecting if `z = -1.0`. If so, it logs a warning, cancels any active goal using the stored `current_goal_handle`, resets the internal state—`goal_in_progress`, `current_goal_handle`, `current_goal`—and returns. If the point is valid, it attempts to transform it from the camera frame to the global map frame using `lookup_transform` and `do_transform_point`. On failure, it returns. It then calls `get_robot_position` to find the robot's current pose in the map frame; if this fails, it returns. With both target and robot positions available, it calls `compute_goal`, storing the calculated goal pose in `next_goal` and getting the straight-line `goal_distance`. It checks if `goal_distance` is below `stopping_distance`; if true, it modifies the position component of `next_goal` to the robot's current position, effectively creating an in-place rotation goal, which allow the robot to always aim at the target, even if it's close to it. The crucial update logic follows: if `goal_in_progress` is `True`, it compares the newly computed `next_goal` with the currently active `current_goal` using `are_goals_different`. If they are too similar, it returns, avoiding a useless update and ensuring smoother motion. If they differ significantly, it requests cancellation via `current_goal_handle.cancel_goal_async` and uses a callback which calls `send_goal` with the new `next_goal` once the cancellation is complete. If no goal was initially in progress, it simply calls `send_goal` with the computed `next_goal`.

The `get_robot_position` method retrieves the robot's pose in the specified frame. It does this by getting the transform from `robot_frame`—`base_link`—to the `target_frame`—`map`—and applying it to the origin point. It returns the resulting `PointStamped` or `None` on failure.

The `compute_goal` method calculates the target pose and the straight-line distance. Given robot and target positions, it computes the distance using `math.hypot` and the required `yaw` using `math.atan2` in order to have a resulting pose where the robot will face the target. The yaw is then converted to a `Quaternion` via the static helper `yaw_to_quaternion` and these values are used to populate a `PoseStamped` message, which is returned along with the computed distance.

The `are_goals_different` helper method compares `old_goal` and `new_goal` to determine if they are significantly different, based on predefined thresholds. First, it handles the edge case where either goal is `None`, returning `True` (indicating they are different). It then calculates the Euclidean distance between the (x, y) positions of the two goals using `math.hypot`. If this distance exceeds `distance_update_threshold`, the function immediately returns `True`. If the distance is within the threshold, it proceeds to compare the orientation. It converts the `Quaternion` orientation of both goals into yaw angles using the `quaternion_to_yaw` static helper. The difference between these yaw angles is calculated and then normalized to the range $[-\pi, \pi]$ using the `normalize_angle` static helper to find the shortest angular distance. The absolute value of this normalized difference is compared against `yaw_update_threshold`. If the yaw difference exceeds this threshold, the function returns `True`. If both the distance and the normalized yaw difference are within their respective thresholds, the function returns `False`, indicating the goals are considered too similar to warrant an update.

Action client interactions are managed via asynchronous calls and callbacks. `send_goal` packages the `PoseStamped` into a `NavigateToPose.Goal` and sends it using `_action_client.send_goal_async`.

It registers `goal_response_callback` to handle the server's acceptance or rejection.

The `goal_response_callback` is triggered when the server responds to the `send_goal_async` request. It checks `goal_handle.accepted`. If rejected, it logs a warning and resets the node's state (`goal_in_progress`, `current_goal_handle`, `current_goal`). If accepted, it logs confirmation, sets `goal_in_progress` to `True`, stores the received `goal_handle` in `current_goal_handle`, and, crucially, updates `self.current_goal` to match the `self.next_goal` that was just sent and accepted. It then calls `goal_handle.get_result_async` and registers `get_result_callback` to handle the final result.

The `get_result_callback` is invoked when the navigation action concludes. It resets the node's state variables, indicating the action is no longer active.

Static helper methods `yaw_to_quaternion`, `quaternion_to_yaw`, and `normalize_angle` provide essential angle conversion and normalization utilities, used in `compute_goal` and `are_goals_different`.

Again, the script includes the usual Python entry point structure.

5.4 Initial position publisher

This utility node, `Initial Position Publisher`, serves a specific purpose during the system startup sequence, particularly when running in simulation. Its function is to provide the Nav2 stack, specifically the AMCL (Adaptive Monte Carlo Localization) node, with an initial estimate of the robot's pose within the map. This is necessary because AMCL requires a starting point to begin the localization process. Without an initial pose estimate, AMCL would typically start with a widely dispersed particle cloud across the entire map, requiring significant movement and sensor data to converge on the robot's true location. This node automates the process of sending this initial estimate.

Logic pipeline

1. Waits until the nav2 is ready.
2. When ready, **publishes the robot initial pose**.
3. **Destroys itself**, since it is no longer needed.

Code analysis

```
1 import rclpy
2 from rclpy.node import Node
3
4 import math
5
6 from geometry_msgs.msg import PoseWithCovarianceStamped, Quaternion
7
8 from nav_msgs.msg import OccupancyGrid
9
10
11 class InitStatePub(Node):
12
13     def __init__(self):
14         super().__init__('initial_state_publisher')
15
16         self.declare_parameter('x', 0.0)
17         self.declare_parameter('y', 0.0)
18         self.declare_parameter('yaw', 0.0)
19
20         self.x = self.get_parameter('x').value
21         self.y = self.get_parameter('y').value
22         self.yaw = self.get_parameter('yaw').value
```

```
23
24     self.publisher_ = self.create_publisher(
25         PoseWithCovarianceStamped,
26         '/initialpose',
27         10
28     )
29
30     # The callback will be triggered only when nav2 is ready, since only
31     # ↪ in this case
32     # messages arrive on the /map topic
33     self.subscriber_ = self.create_subscription(
34         OccupancyGrid,
35         '/map',
36         self.publish_pose,
37         10
38     )
39
40     self.get_logger().info(f'Waiting for Nav2 to be ready...')
41
42     def publish_pose(self, placeholder_msg):
43         '''
44         This function publishes the initial pose based on the received
45         ↪ parameters,
46         then calls a timer which triggers the node destruction procedure
47         '''
48         # Destroy subscriber to avoid running again this callback
49         self.destroy_subscription(self.subscriber_)
50
51         msg = PoseWithCovarianceStamped()
52         msg.header.frame_id = 'map'
53         msg.header.stamp = self.get_clock().now().to_msg()
54         msg.pose.pose.position.x = self.x
55         msg.pose.pose.position.y = self.y
56         q = self.yaw_to_quaternion(self.yaw)
57         msg.pose.pose.orientation = q
58
59         self.publisher_.publish(msg)
60
61         self.get_logger().info(f'Initial pose published')
62
63         self.timer = self.create_timer(2.0, self.shutdown_node)
64
65     def shutdown_node(self):
66
67         self.get_logger().info(f'Mission accomplished, destroying node')
68
69         self.destroy_node()
70         rclpy.shutdown()
71
72     @staticmethod
73     def yaw_to_quaternion(yaw):
74         '''
75         This function computes the quaternion for robots moving on a plane,
76         ↪ given the yaw angle
77         '''
78         q = Quaternion()
```

```
76     q.w = math.cos(yaw / 2.0)
77     q.z = math.sin(yaw / 2.0)
78     q.x = 0.0
79     q.y = 0.0
80     return q
81
82
83
84
85 def main(args=None):
86     rclpy.init(args=args)
87     node = InitStatePub()
88     rclpy.spin(node)
89
90
91 if __name__ == '__main__':
92
93     main()
```

The class **constructor** initializes the `initial_state_publisher` node. It declares ROS 2 parameters 'x', 'y', and 'yaw' with default values of 0.0, allowing the initial pose to be configured externally—e.g., via command line arguments or in a launch file. It then retrieves these parameter values using `self.get_parameter().value`. A publisher is created to send `PoseWithCovarianceStamped` messages to the standard `/initialpose` topic used by AMCL. Crucially, instead of publishing immediately, it creates a subscriber to the `/map` topic. This subscription acts as a one-time trigger; its callback, `publish_pose`, will only execute once a map message is received, indicating that the nav2 nodes are ready.

The `publish_pose` method is the core logic, executed upon receiving the first message on `/map`. It immediately destroys the map subscriber using `self.destroy_subscription()` to ensure it only runs once. It then constructs the `PoseWithCovarianceStamped` message, setting the header's `frame_id` to 'map' and the timestamp to the current time. The position is populated using the `x` and `y` parameters retrieved during initialization. The orientation is calculated by converting the `yaw` parameter using the helper function `yaw_to_quaternion`. The covariance matrix is left at its default zero values, meaning high certainty in the initial estimate. The constructed message is then published using `publisher.publish(msg)`. After publishing, it starts a one-shot **Timer** set for 2.0 seconds, which will call the `shutdown_node` method upon expiration.

The `shutdown_node` method is a simple cleanup function called by the timer. It logs a message and then calls `destroy_node()` to release the node's resources, effectively stopping its execution since its single task—publishing the initial pose—is complete.

Finally, the script includes the standard Python entry point.

6 Launch File

The whole project can be run using the Python launch file called `doom_project.launch.py` included in the `doom_nodes` package. It serves as the primary entry point to start the entire robot system simulation, including the environment, the navigation stack, and the custom nodes developed for this project. It uses ROS 2 launch actions to include other launch files and start individual nodes with specific configurations.

```
1 from launch import LaunchDescription
2 from launch_ros.actions import Node
3 from launch.actions import IncludeLaunchDescription
4 from launch.launch_description_sources import PythonLaunchDescriptionSource
5 from ament_index_python.packages import get_package_share_directory
6 import os
7
8
9 def generate_launch_description():
10     return LaunchDescription([
11         IncludeLaunchDescription(
12             PythonLaunchDescriptionSource(
13                 os.path.join(
14                     get_package_share_directory('turtlebot3_gazebo'),
15                     'launch',
16                     'turtlebot3_world.launch.py'
17                 )
18             ),
19             launch_arguments={ 'use_sim_time': 'true' }.items()
20         ),
21         IncludeLaunchDescription(
22             PythonLaunchDescriptionSource(
23                 os.path.join(
24                     get_package_share_directory('turtlebot3_navigation2'),
25                     'launch',
26                     'navigation2.launch.py'
27                 )
28             ),
29             launch_arguments={
30                 'use_sim_time': 'true',
31                 'map': 'saved_maps/hexa_world.yaml'
32             }.items()
33         ),
34         Node(
35             package='doom_nodes',
36             executable='perceptor',
37             name='perceptor',
38             output='log',
39             parameters=[{'use_sim_time': True}]
40         ),
41         Node(
42             package='doom_nodes',
43             executable='localizer',
44             name='localizer',
45             output='log',
46             parameters=[{'use_sim_time': True}]
47         ),
48         Node(
```

```
49     package='doom_nodes',
50     executable='initial_state_publisher',
51     name='initial_state_publisher',
52     output='screen',
53     parameters=[{'use_sim_time': True}]
54 ),
55 Node(
56     package='doom_nodes',
57     executable='navigator',
58     name='navigator',
59     output='screen',
60     parameters=[{'use_sim_time': True}]
61 )
62 ]]
```

The launch file defines a `generate_launch_description` function, which is the standard entry point recognized by the ROS 2 launch system. This function returns a `LaunchDescription` object containing a list of actions to be executed.

First, it uses `IncludeLaunchDescription` to incorporate the standard launch file for the TurtleBot3 Gazebo world `turtlebot3_world.launch.py`. The `get_package_share_directory` function is used to locate the launch file within the installed `turtlebot3_gazebo` package.

Next, another `IncludeLaunchDescription` action is used to bring up the entire Nav2 stack by including `navigation2.launch.py` from the `turtlebot3_navigation2` package. The `'map'` argument is provided, specifying the path to the pre-saved map file `hexa_world.yaml` that Nav2 should load for localization and planning.

Following the inclusion of the simulation and navigation stacks, four `Node` actions are defined to launch the custom nodes developed for this project: `perceptor`, `localizer`, `initial_state_publisher`, and `navigator`. Each `Node` action specifies:

- **package:** The name of the ROS 2 package containing the node, in our case `'doom_nodes'`.
- **executable:** The name of the Python script to execute.
- **name:** The name assigned to the node.
- **output:** Where the node's standard output, like logs, should be directed. The two choices are `'screen'` to display output in the terminal where the launch file was started, or `'log'` to send it only to ROS 2 log files, making it not visible in the terminal.
- **parameters:** A list of parameters to pass to the node. Here, `{'use_sim_time': True}` is passed to all custom nodes to ensure they synchronize with the simulation clock. The `initial_state_publisher` initial pose parameters (x, y, yaw) could also be passed here if needed, overriding the defaults declared within the node.

When this launch file is executed using `ros2 launch doom_nodes doom_project.launch.py`, it systematically brings up the Gazebo simulation, the Nav2 stack configured with the specified map, and all the necessary custom nodes, ensuring all components are clocked by the simulation clock.

7 Video link

A video demonstration showcasing the complete system functioning within the simulation can be accessed via the following link:

INSERT LINK

8 Conclusion and Future Work

This project successfully demonstrated the development and integration of a ROS 2 based system for dynamic human target following using a mobile robot equipped with an RGB-D camera.

The system architecture relies on a **modular design**, dividing responsibilities among distinct nodes, which enhances maintainability and potential reusability of the components.

By combining real-time object detection with YOLO, 3D localization using depth data, and intelligent goal management interacting with the Nav2 stack, the system enables the robot to autonomously approach and track a designated person within a simulated environment containing obstacles. Key features include the localization of the target in 3D using combined RGB and depth camera data, robust handling of target loss via error signal propagation, and logic to prevent overly frequent goal updates, leading to smoother pursuit behavior.

The current system's performance, particularly its ability to track very rapid target movements, is primarily constrained by the processing speed of the perception pipeline and the update frequency of the navigation commands. The simulated camera frame rate and the *Navigator* node's timer frequency—currently set at 2 Hz and 1 Hz respectively—were chosen to ensure real-time operation within the available computational resources. With access to more powerful hardware—e.g. a higher-performance GPU for YOLO inference and a faster CPU—these rates could be significantly increased. A higher camera frame rate would provide more frequent updates on the target's position, while a faster **navigate** loop in the *Navigator* node would allow the robot to react more quickly to these updates, enabling the pursuit of faster-moving targets.

Looking ahead, several enhancements could further improve the system's robustness and capabilities. A primary limitation of the current setup is the camera's fixed field of view. If the target moves quickly to the side or behind the robot, it can be lost. Integrating a 360-degree camera or multiple strategically placed cameras could provide a complete omnidirectional view, significantly mitigating the issue of losing the target due to FOV constraints and allowing for more continuous tracking. Further work could also explore more sophisticated prediction algorithms within the to anticipate the target's future movements, potentially leading to even smoother and more proactive following behavior.

The developed dynamic human-following system holds significant potential across various domains. Its core capability—enabling a robot to autonomously follow a specific person while navigating obstacles—can be adapted for numerous practical applications. For instance, in **industrial logistics and assistance**, the robot could be integrated with a cart or carrying platform. Workers could then have the robot follow them, transporting tools, components, or finished goods, thereby reducing physical strain and improving workflow efficiency, such as a technician being followed by a robot carrying their heavy toolkit. Furthermore, the system could serve as an assistive robot for **personal assistance and healthcare**, aiding the elderly or individuals with mobility impairments by following them and carrying essential items like medication or personal belongings, potentially also acting as a mobile telepresence unit. Across all applications, usability could be significantly improved by integrating **voice command recognition**, allowing workers or users to interact naturally using instructions to control the robot behaviour, without needing manual controls. These applications leverage the robot's ability to stay close to a person autonomously, freeing the human from directly controlling the robot's movement and allowing them to focus on their primary tasks.